

FEG Prism

Kuratowski Calculus Engine — Code Manual

Juan Pablo Silva Alvarado

February 24, 2026

Part of Modulo Synthesis / Fractal Entropic Geometrodynamics

DOI: 10.5281/zenodo.18746995

*“The universe is strictly computable, finite, and structurally
homeostatic. The rest is just counting.”*

Contents

1	Theory	1
1.1	Fractal Entropic Geometrodynamics	1
1.2	The Three Geometries	1
1.3	Key Invariants	2
2	Architecture	3
2.1	Pipeline Overview	3
2.2	Module Map	3
2.3	Execution Modes	4
3	Phase 1 — Poisson Sprinkling	5
3.1	The Causal Diamond	5
3.2	Rejection Sampling	5
3.3	Streaming Chunks	5
4	Phase 1 — Hasse Diagram	6
4.1	Transitive Reduction	6
4.2	Tier 1: Sparse A^2 Reduction	6
4.3	Tier 2: Geometric OCI Construction	6
4.3.1	Adaptive Termination Valves	6
4.4	Bulk Momentum	7
5	Graph Infrastructure	8
5.1	Type-Safe CSR	8
5.2	Grid Quantisation	8
6	Phase 2 — Kuratowski Contraction	10
6.1	Causal Prism Detection	10
6.1.1	Detection Algorithm	10
6.2	K_5 Threat Contraction	10
6.3	Generation Classification	11
6.4	Phase-Coherence Decomposition	11
6.5	Topology Summary	11
7	Phase 3 — Spectral Observables	12
7.1	Spectral Dimension	12
7.2	Tier 1: Eigendecomposition	12
7.3	Tier 2: Monte Carlo Walkers	12
7.4	Causal Flux	13
7.5	Diffusion Steps	13
8	Ensemble Framework	14
8.1	Ensemble Runner	14
8.2	Seed Derivation	14
8.3	Concurrency	14
8.4	Checkpointing	14
8.5	Averaging	15

9	Measurements M1–M5	16
9.1	M1: Traversal Mass Ratios	16
9.2	M2: Half-Life Census	16
9.3	M3: Modulo Path Integral (NTT)	16
9.4	M4: Vacuum Polarisation	17
9.5	M5: Electroweak Sector	17
10	Measurements M6–M10	18
10.1	M6: Quantum Decoherence	18
10.2	M7: Neutrino Census	18
10.3	M8: PMNS Mixing Matrix	18
10.4	M9: Higgs Mechanism	19
10.5	M10: Standard Model Lagrangian Card	19
11	Output & CSV	20
11.1	CSV Writer	20
11.2	Spectral CSV	20
11.3	Topology CSV	20
11.4	Measurement CSVs	21
11.5	Terminal Summary	21
11.6	Metadata	21
12	Animation Export	22
12.1	Topology Slice	22
12.2	Usage	22
13	Configuration	23
13.1	Configuration File	23
13.2	Config Struct	23
13.3	Memory Estimation	23
13.4	Mode Selection Flow	23
14	Provenance	24
14.1	Architecture	24
14.2	Preimage and Hash	24
14.3	Custom SHA-256	24
14.4	Build-Time Embedding	24
14.5	UTC Timestamp	25
15	Usage & CLI Reference	26
15.1	Synopsis	26
15.1.1	Positional Arguments	26
15.2	Flags	26
15.2.1	Execution Mode	26
15.2.2	Physics Parameters	26
15.2.3	Ensemble Control	26
15.2.4	Measurements	27
15.2.5	Modulo Configuration	27
15.2.6	Export	27
15.3	Quick-Start Examples	27
15.3.1	Minimal Run	27
15.3.2	Full Standard Model Card	27
15.3.3	Production Run	27

15.3.4	Born Rule Verification	28
15.3.5	PMNS Mixing Matrix	28
15.3.6	Higgs Mass Estimate	28
15.3.7	Animation Export	28
15.3.8	Resume from Checkpoint	28
15.3.9	Custom Modulo Parameters	28
16	Expected Results & Verification	29
16.1	Spectral Dimension	29
16.2	Mass Hierarchy	29
16.3	Fine Structure Constant	29
16.4	Dark Matter Fraction	29
16.5	Born Rule	30
16.6	Verification Checklist	30
16.7	Known Finite-Size Effects	30
17	Dependencies	31
17.1	Crate List	31
17.2	Build Profile	31
17.3	Notable Absences	31
	Colophon	32

Theory

Chapter Overview

- *The physical framework: Fractal Entropic Geometrodynamics (FEG)*
- *Three Geometries of spacetime, matter, and logic*
- *Key invariants enforced by the simulation*

1.1 Fractal Entropic Geometrodynamics

FEG Prism is a Monte Carlo engine implementing the Kuratowski Calculus — the mathematical framework underlying *Fractal Entropic Geometrodynamics* (FEG) by Juan Pablo Silva Alvarado (DOI: 10.5281/zenodo.18746995). The central claim of FEG is that four-dimensional spacetime, the Standard Model particle spectrum, and quantum mechanics all emerge from a single discrete structure: the Hasse diagram of a Poisson-sprinkled causal set, constrained only by **planarity preservation** (Kuratowski’s theorem).

The engine is named after its fundamental topological particle: the **Causal Prism** $K_{2,N}$, a complete bipartite subgraph with two poles and $N \geq 3$ intermediaries. These are the only non-trivial substructures permitted by the triangle-free property of the transitive reduction.

1.2 The Three Geometries

Definition 1: Geometry of Order — Phase 1

Poisson sprinkling into a 4D causal diamond produces N events. The **transitive reduction** (Hasse diagram) removes all redundant causal links: edge (u, v) survives iff no intermediate w satisfies $u \prec w \prec v$. The result is a triangle-free directed acyclic graph (DAG) whose Lorentz invariance is emergent from the isotropy of the Poisson process.

Definition 2: Geometry of Matter — Phase 2

The **Causal Prism** $K_{2,N}$ is the unique non-trivial bipartite subgraph permitted in a triangle-free graph. It consists of:

- Two **poles** (origin u , destination v) at graph-distance ≥ 2
- $N \geq 3$ **intermediaries** $w_1, \dots, w_N$ shared by both poles
- Intermediaries are mutually disconnected (guaranteed by triangle-freeness)

Topological mass is the integer $M = N$ (number of intermediaries). The mass hierarchy reduces to $N \in \{3, 4, 5+\}$ for electron, muon, tau.

Definition 3: Geometry of Logic — Computational Complexity

The 2-hop detection algorithm for Causal Prisms runs in $O(N)$ because the physics is local. Hasse links have bounded proper time ($\tau \leq 8$); therefore $K_{2,N}$ poles must be within bounded causal distance. The bounded degree ($\text{MAX_HASSE_DEGREE} = 15$) ensures $O(1)$ per-node work. This is not merely an optimisation — it is a computational proof that **physical locality is an information principle**.

1.3 Key Invariants

The following invariants are preserved throughout the simulation pipeline:

1. **Triangle-freeness:** The Hasse diagram contains no 3-cliques. Any triangle $u \rightarrow w \rightarrow v$ with $u \rightarrow v$ would be transitively reduced. This is a mathematical theorem, not an empirical check.
2. **Planarity preservation:** K_5 threats are absorbed into the nearest pole via vertex contraction, enforcing Kuratowski's planarity criterion. This is the discrete analogue of colour confinement.
3. **Lorentz invariance:** No spatial cutoff is applied during Hasse construction. The light cone search at time offset Δt uses radius $\Delta t + 1$ (natural units $c = 1$), preserving exact causal structure.
4. **Strict finitism:** All accumulations use u64 integers. The single f64 division occurs only at the final return. No floating-point drift across ensemble realisations.
5. **Deterministic reproducibility:** Given the same seed, the engine produces identical output on the same platform. Seeds are derived via `StdRng::seed_from_u64(seed_base + k)` for realisation k .

Architecture

Chapter Overview

- *Six-phase simulation pipeline*
- *Module organisation and data flow*
- *Execution modes: in-memory vs. streaming*

2.1 Pipeline Overview

The simulation proceeds through six phases, each corresponding to a Rust module:

```

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
X Phase 1: Sprinkle + Hasse          (phase1/)                      X
X   Poisson sprinkling → Transitive Reduction (CSR)                X
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
X Phase 2: Kuratowski Contraction  (phase2/)                      X
X   K_{2,N} detection → K_5 threat absorption → Defect            X
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
X Phase 3: Spectral Observables    (phase3/)                      X
X   Random walkers → d_S(t) → Causal Flux                        X
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
X Measurements M1-M10              (measure/)                     X
X   Mass, half-life, modulo, vacuum pol., electroweak,            X
X   decoherence, neutrino, PMNS, Higgs, Lagrangian                X
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
X Phase 4: Output                  (output/)                      X
X   CSV serialisation + provenance stamps                          X
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
X Ensemble Runner                  (ensemble/)                   X
X   Adaptive Welford convergence + checkpointing                  X
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

```

2.2 Module Map

Module	Responsibility
<code>main.rs</code>	CLI parsing, flag dispatch, provenance boot
<code>config.rs</code>	TOML configuration, RAM-aware mode selection
<code>provenance.rs</code>	Five-layer SHA-256 provenance system
<code>phase1/</code>	Poisson sprinkling (<i>sprinkle.rs</i>) and Hasse construction (<i>hasse.rs</i>)
<code>phase2/</code>	Causal Prism detection (<i>defect.rs</i>), topology summary (<i>topology.rs</i>)
<code>phase3/</code>	Spectral dimension (<i>spectral.rs</i>), random walkers (<i>walker.rs</i>), causal flux (<i>flux.rs</i>)
<code>graph/</code>	Type-safe CSR (<i>csr.rs</i>), grid quantisation (<i>grid.rs</i>)

ensemble/	Adaptive batching (<i>runner.rs</i>), Welford averaging (<i>averaging.rs</i>), checkpointing (<i>checkpoint.rs</i>)
measure/	Ten observer measurements M1–M10, each in <i>m01_traversal.rs</i> through <i>m10_lagrangian.rs</i>
output/	CSV writer (<i>csv.rs</i>), human-readable summary (<i>summary.rs</i>)
anim_export.rs	Binary topology slice for Manim animation

2.3 Execution Modes

Definition 1: In-Memory Mode

All N points and the full CSR adjacency are held in RAM. This is the fast path. Memory estimate: $N \times 2000$ bytes for $N \leq 50,000$; $N \times 500$ bytes for $N > 50,000$ (Hasse degree bounded by 15). Selected by `--inmemory` or recommended automatically when estimated memory fits within 70% of available RAM.

Definition 2: Streaming Mode

For extreme scales ($N > 10^7$), the simulation uses time-slab sprinkles and a sliding window. Only the active window is held in RAM. Estimate: $N \times 13 + 100$ MB overhead. Selected by `--stream`.

The mode is auto-detected by `config::recommend_mode()`, which queries `sysinfo::System` for available memory and applies the safety fraction from *causal_set.toml*.

Phase 1 — Poisson Sprinkling

Chapter Overview

- *Sprinkling N events into a 4D causal diamond*
- *Volume formula and density normalisation*
- *Streaming chunks for sliding-window mode*

3.1 The Causal Diamond

Source: `phase1/sprinkle.rs`

The simulation domain is a 4D causal diamond: the intersection of the future light cone of a past tip p^- and the past light cone of a future tip p^+ . In Cartesian coordinates, the constraint is

$$|t| + r \leq T/2, \quad r = \sqrt{x^2 + y^2 + z^2}.$$

The diamond has 4-volume $V = \pi T^4/24$. To normalise the Poisson density to $\rho \approx 1$, we set

$$T = \left(\frac{24N}{\pi} \right)^{1/4}.$$

3.2 Rejection Sampling

The function `sprinkle(n, rng)` draws uniform random points in the bounding box $[-T/2, T/2]^4$ and accepts those satisfying $|t| + r \leq T/2$. The acceptance rate is $\pi/24 \approx 13\%$ in 4D. The returned array has exactly N points in $[t, x, y, z]$ format.

3.3 Streaming Chunks

For the sliding-window path, `sprinkle_chunk()` generates points in a time slab $[t_{\min}, t_{\max}]$, returning `GridPoint` structs with sequential `orig_idx` numbering. Grid quantisation fields (`qt`, `qx`, `qy`, `qz`) are filled by the caller after choosing grid dimensions.

Phase 1 — Hasse Diagram

Chapter Overview

- *Two-tier Hasse construction: sparse A^2 vs. geometric OCI*
- *Transitive reduction as the origin of triangle-freeness*
- *Adaptive early termination valves*

4.1 Transitive Reduction

Source: `phase1/hasse.rs`

Given N Poisson-sprinkled events with coordinates $p_i = (t_i, x_i, y_i, z_i)$, two events are **causally related** iff

$$(t_j - t_i)^2 - |\vec{x}_j - \vec{x}_i|^2 > 0, \quad t_j > t_i.$$

The Hasse diagram is the **transitive reduction**: edge (i, j) exists iff $i \prec j$ and there is no intermediate k with $i \prec k \prec j$. This guarantees triangle-freeness by construction — any triangle would imply a transitively redundant edge.

4.2 Tier 1: Sparse A^2 Reduction

For $N \leq 15,000$, the function `build_hasse_sparse()` materialises the full causal closure as a sparse matrix A (via `sprs::TriMat`), computes A^2 , and retains only edges where $A^2[i, j] = 0$ (no 2-hop path exists). This is exact but $O(N^2)$ in space.

Theorem 1: Sparse Reduction Correctness

In a transitively closed DAG, edge (i, j) is a Hasse link if and only if $A^2[i, j] = 0$.
Proof: if a 2-hop path $i \rightarrow k \rightarrow j$ exists, then j is reachable from i via k , making the direct edge redundant.

4.3 Tier 2: Geometric OCI Construction

For $N > 15,000$, the function `build_hasse_direct()` avoids materialising the full closure. Instead:

1. **Quantise & sort**: Coordinates are quantised to an integer lattice via `quantize_and_sort()`, yielding `GridPoint` records sorted by (q_t, q_x, q_y, q_z) for cache locality.
2. **Build OCI**: The Occupied Cell Index (OCI) is a per-layer structure mapping (q_x, q_y, q_z) cells to the points they contain, sorted by q_x for binary search.
3. **Parallel link discovery**: For each source u , iterate over Δt shells from 0 to `MAX_CAUSAL_DEPTH`. For each shell, query the OCI for cells within the light cone ($r_{\max} = \Delta t + 1$), test causality, and check transitivity against existing children.
4. **CSR assembly**: Collect per-chunk degrees and targets, build the CSR head and data arrays, sort each neighbour list for binary-search correctness.

4.3.1 Adaptive Termination Valves

Three early-exit conditions accelerate the search:

1. **Degree cap** (`MAX_HASSE_DEGREE = 15`): A node with ≥ 15 descendants has a fully populated causal shadow. Further links are redundant.

2. **Alexandrov volume short-circuit:** If every causal candidate in a Δt shell is transitively blocked (non-empty Alexandrov set between u and v), further shells have strictly larger Alexandrov volume and will also be blocked.
3. **Consecutive dry shells:** Two consecutive shells with no causal candidates plus ≥ 2 existing descendants implies the light cone slice is empty.

4.4 Bulk Momentum

Both tiers compute the **bulk momentum** vector: $p_i = \text{out-degree}(i) - \text{in-degree}(i)$. This quantity encodes the net causal asymmetry at each node and is used in Phase 2 for generation classification ($\phi(w) = \text{sign}(p_w) \in \{-1, 0, +1\}$).

Graph Infrastructure

Chapter Overview

- *Type-safe CSR with compile-time direction marker*
- *Grid quantisation and the Occupied Cell Index*
- *Physical constants encoded as Rust constants*

5.1 Type-Safe CSR

Source: `graph/csr.rs`

The central data structure is `CsrGraph<D>`, a Compressed Sparse Row graph parametrised by a zero-sized direction marker:

- `CsrGraph<Directed>`: forward-only edges ($u \rightarrow v$ where $t_u < t_v$). Produced by Phase 1.
- `CsrGraph<Undirected>`: both directions stored explicitly. Produced by `symmetrize()` or the defect graph.

The `PhantomData<D>` trick prevents passing a directed Hasse diagram to spectral walkers (which require the symmetric graph) at *compile time*.

Definition 1: CSR Storage Layout

`head[u]..head[u+1]` indexes into data to give the sorted neighbour list of node u . Binary search on sorted neighbours gives $O(\log d)$ edge queries, where $d \leq 15$ in practice.

Key operations:

- `neighbors(u)`: returns `&[u32]` slice of sorted neighbours
- `degree(u)`: returns out-degree (or total degree for undirected)
- `has_edge(u, v)`: binary search on sorted neighbour list
- `symmetrize()`: converts `Directed` $\rightarrow$ `Undirected` by adding reverse edges (method on `CsrGraph<Directed>` only)
- `reverse()`: reverses all edge directions (parents $\leftrightarrow$ children)
- `unique_edges()`: extracts (u, v) pairs with $u < v$ for eigendecomposition (method on `CsrGraph<Undirected>` only)

Free functions:

- `build_directed_csr(n, rows, cols)`: builds from unsorted edge pairs
- `build_undirected_csr(n, rows, cols)`: builds symmetric CSR from edge pairs with automatic reverse insertion

5.2 Grid Quantisation

Source: `graph/grid.rs`

The OCI construction requires mapping continuous coordinates to an integer lattice. The `GridPoint` struct stores both the original `[f64; 4]` coordinates and quantised `(qt, qx, qy, qz)` fields. The `GridDims` struct encodes lattice dimensions derived from coordinate bounds.

Key constants from `grid.rs`:

- `MAX_HASSE_DEGREE = 15`: maximum out-degree in the Hasse diagram

- MAX_CAUSAL_DEPTH: maximum Δt shells to search
- MAX_PROPER_TIME_SQ: proper-time cutoff for causality

Phase 2 — Kuratowski Contraction

Chapter Overview

- Causal Prism ($K_{2,N}$) detection via 2-hop traversal
- K_5 threat contraction for planarity preservation
- Generation classification by bulk-momentum phase signature
- Phase-coherence decomposition: visible, dark, sterile mass

6.1 Causal Prism Detection

Source: `phase2/defect.rs`

The Causal Prism is the fundamental topological particle in FEG. It is a complete bipartite subgraph $K_{2,N}$ embedded in the triangle-free Hasse DAG:

Definition 1: Causal Prism

A **Causal Prism** consists of:

- An **origin** u (past pole) and **destination** v (future pole) at graph-distance ≥ 2
- $N \geq 3$ **intermediaries** $w_1, \dots, w_N$ such that $u \rightarrow w_i$ and $w_i \rightarrow v$ for all i
- The intermediaries are mutually disconnected (guaranteed by the triangle-free property)

The minimum $N = 3$ (constant `MIN_PRISM_SHARED`) derives from the Uniqueness of Bifurcation-Convergence theorem: $K_{2,2}$ yields only $S_2 \rightarrow U(1)$; the $SU(3)$ gauge structure demands S_3 , hence $N \geq 3$.

6.1.1 Detection Algorithm

The 2-hop forward-forward traversal:

1. Select a **core** of the top 10% of nodes by undirected degree (out-degree + in-degree from the reverse CSR).
2. For each core node u (not yet placed):
 - (a) Collect 2-hop candidates: for each child w of u , for each child v of w that is also in the core and not yet placed, add v to the candidate set.
 - (b) For each candidate v : compute $\text{belly} = \text{children}(u) \cap \text{parents}(v)$ using sorted-slice intersection ($O(d_u + d_v)$).
 - (c) Select the best partner v^* with maximum $|\text{belly}|$.
 - (d) If $|\text{belly}| \geq 3$: commit the prism, mark all component nodes as placed.

Key Result

The algorithm is $O(N \cdot D^2)$ where $D = \max \text{Hasse degree} \leq 15$. Since $D = O(1)$, the total complexity is $O(N)$.

6.2 K_5 Threat Contraction

After prism detection, external nodes threatening to form a K_5 minor are absorbed:

Definition 2: K_5 Threat Criterion

An external node t is a **threat** to prism $(u, v, \{w_i\})$ if:

- t is connected (undirected) to *both* poles u and v
- t is connected to ≥ 2 intermediaries (constant PRISM_THREAT)

The five nodes $\{u, v, t, w_i, w_j\}$ are then mutually reachable, forming a K_5 minor.

Contraction: t is merged into the higher-degree pole.

Merge chains are resolved transitively: if $t_1 \rightarrow t_2 \rightarrow u$, then t_1 maps directly to u . The resulting merge_map is a forest of stars.

6.3 Generation Classification

Each intermediate w_i carries a **causal phase** $\phi(w_i) = \text{sign}(\text{bulk_momentum}[w_i]) \in \{-1, 0, +1\}$.

Definition 3: Generation Number

The **generation** of a prism P is $g(P) = |\{\phi(w_i) : w_i \in \text{intermediaries}\}|$, i.e., the number of distinct phase classes. Since $\phi \in \{-1, 0, +1\}$, $g \in \{1, 2, 3\}$.

Definition 4: Net Phase and Matter/Antimatter

The **net phase** $\Phi(P) = \sum_i \phi(w_i)$.

- $\Phi > 0$: **matter** (e.g., electron for $g = 1$)
- $\Phi < 0$: **antimatter** (e.g., positron for $g = 1$)
- $\Phi = 0$: **sterile** (dark matter candidate)

The generation sets (GenerationSets) store node indices for Gen1, Gen2, Gen3, Anti-Gen1, and Sterile prisms. Average topological mass per generation is $\langle M \rangle_g = \langle N \rangle$ (average intermediate count).

6.4 Phase-Coherence Decomposition

The defect module computes a decomposition of gravitational mass into visible and dark components:

$$M_{\text{grav}}(P) = N, \quad (6.1)$$

$$M_{\text{vis}}(P) = |\Phi(P)|, \quad (6.2)$$

$$M_{\text{dark}}(P) = N - |\Phi(P)|. \quad (6.3)$$

Aggregated ratios:

- $\Omega_{\text{dark}}/\Omega_{\text{vis}} = \sum(N - |\Phi|)/\sum|\Phi|$
- $Q_{\text{topo}} = \sum|\Phi|^2/\sum N^2$ (topological charge ratio, intrinsic)
- $\alpha_{\text{EM}} = Q_{\text{topo}}/(8\pi)$ (observed electromagnetic coupling)

6.5 Topology Summary

Source: *phase2/topology.rs*

The TopologySummary struct aggregates 27 fields including total nodes/prisms, per-generation counts and masses, phase census, Q_{topo} , α_{EM} , and the prism histogram (belly-size distribution).

Phase 3 — Spectral Observables

Chapter Overview

- Spectral dimension $d_S(t)$ from random walks
- Two computation tiers: eigendecomposition vs. Monte Carlo walkers
- Causal flux: attraction and repulsion between generations

7.1 Spectral Dimension

Source: `phase3/spectral.rs`

The spectral dimension quantifies the effective dimensionality experienced by a diffusing particle at scale t :

$$d_S(t) = -2 \frac{d \ln P(t)}{d \ln t},$$

where $P(t)$ is the return probability after t diffusion steps on the graph. Centred finite differences are used for interior points; forward/backward differences at endpoints.

Definition 1: WalkResult

Each walk category (vacuum, defect, per-generation, sterile, flux) returns:

- `p`: $P(t)$ return probability at each measurement step
- `ds`: $d_S(t)$ spectral dimension derived from $P(t)$
- `ds_std`: ensemble standard deviation (empty for $M = 1$)

Definition 2: SpectralOutput

The composed output struct groups:

- Vacuum (global + core) and defect (global + core) walks
- Per-generation walks: [Gen1, Gen2, Gen3, Anti1]
- Sterile walk (phase-cancelled prisms)
- Causal flux: attraction (Gen1→Anti1) and repulsion (Gen1→Gen1)
- Normalised flux: flux per target node
- Average topological mass per generation

7.2 Tier 1: Eigendecomposition

For $N \leq \text{eigen_cutoff}$ (default 3000), the function `compute_eigen()` computes the full eigendecomposition of the graph Laplacian $L = D - A$ via `nalgebra`. The return probability is then

$$P(t) = \frac{1}{N} \sum_{k=0}^{N-1} e^{-\lambda_k t},$$

where λ_k are the eigenvalues of the normalised Laplacian. Core walks restrict the sum to eigenvectors with support on core nodes.

7.3 Tier 2: Monte Carlo Walkers

Source: `phase3/walker.rs`

For $N > \text{eigen_cutoff}$, random walkers simulate diffusion. The number of walkers W is set by the **Causal Resolution Theorem**:

$$W = \lceil (t_{\max}/\varepsilon)^2 \rceil,$$

where ε is the topological error tolerance (default 0.01) and $t_{\max}$ is the maximum diffusion time (default 15). At $\varepsilon = 0.01$, $t_{\max} = 15$: $W = 2,250,000$ walkers.

Each walker starts at a random node, takes t steps along random edges of the symmetric CSR, and records whether it returns to its origin. Return counts are accumulated as u64 integers (strict finitism).

7.4 Causal Flux

Source: *phase3/flux.rs*

The causal flux measures directed transmission between generation sets:

- **Attraction** (Gen1→AntiGen1): flux from matter to antimatter (opposite topological charge → attraction)
- **Repulsion** (Gen1→Gen1): flux from matter to matter (same charge → repulsion)

Normalised flux divides by the number of target nodes, isolating the intrinsic coupling per unit charge.

7.5 Diffusion Steps

The measurement times are densely sampled near $t = 1$:

```
steps = (1..=30).chain((16..=50).map(|i| i * 2))
```

giving steps 1–30 (unit spacing) and 32, 34, ..., 100 (stride 2).

Ensemble Framework

Chapter Overview

- *Adaptive batching with Welford convergence*
- *Checkpointing and resume capability*
- *Seed derivation for deterministic reproducibility*

8.1 Ensemble Runner

Source: `ensemble/runner.rs`

Each ensemble run consists of M independent Monte Carlo realisations of the full Phase 1–3 pipeline. The runner uses **adaptive Welford convergence**:

1. Run `min_ensemble` realisations (default 8) unconditionally.
2. After each subsequent batch, compute the online running mean and standard error of `mass_gen1` via the Welford algorithm.
3. If the relative standard error $SE/\bar{x} < \text{target_error}$ (default 0.01), declare convergence and stop.
4. Otherwise, continue up to `max_ensemble` (default 50).

Definition 1: EnsembleResult

The output contains:

- `spectral`: Ensemble-averaged `SpectralOutput`
- `topology`: Aggregated `TopologySummary`
- `measurements`: Aggregated `MeasureResults` (if any active)
- `actual_m`: Number of realisations actually run
- `converged`: Whether the target error was reached

8.2 Seed Derivation

Realisation k uses seed `seed_base + k`. The base seed is either user-specified (`--seed`) or derived from the system clock. The deterministic seeding ensures reproducibility: given the same (`seed`, N , M), the output is identical on the same platform.

8.3 Concurrency

Realisations within a batch run in parallel via `rayon`. The maximum concurrency is:

- `min(batch_size, max_concurrent_runs)`
- `max_concurrent_runs`: for $N > 10^7$, limited to 2; otherwise 4
- Overridable via `--threads` and `--batch-size`

8.4 Checkpointing

Source: `ensemble/checkpoint.rs`

After each realisation, `spectral` and `topology` results are serialised to a checkpoint file in the output directory. The `--resume` flag reloads the checkpoint and continues from the last completed realisation. Provenance verification prevents cross-fork resume (Layer 4 of the provenance system).

8.5 Averaging

Source: *ensemble/averaging.rs*

The `average_ensemble()` function computes element-wise means of $P(t)$ and $d_S(t)$ vectors across M realisations, plus ensemble standard deviations. Topology fields are aggregated by `aggregate_topology()`.

Measurements M1–M5

Chapter Overview

- *M1: Traversal mass ratios*
- *M2: Half-life census*
- *M3: Modulo path integral (NTT)*
- *M4: Vacuum polarisation*
- *M5: Electroweak sector*

Source: `measure/mod.rs`

The measurement system provides ten independent physics observables. Each measurement is gated by a boolean flag in `MeasureFlags` and receives a `MeasureContext` containing the full Phase 1–2 outputs. Results are aggregated across ensemble realisations via per-measurement `aggregate()` functions.

9.1 M1: Traversal Mass Ratios

Source: `measure/m01_traversal.rs`

Flag: `--measure-mass`

Measures the **topological inertia** of Causal Prisms by confining random walkers to prism components and recording cover times. The ratio of cover times between generations gives the mass hierarchy:

$$\frac{m_\mu}{m_e} \approx \frac{\langle T_{\text{cover}} \rangle_{\text{Gen2}}}{\langle T_{\text{cover}} \rangle_{\text{Gen1}}}.$$

Output: `traversal_mass.csv` with per-generation cover times and ratios.

9.2 M2: Half-Life Census

Source: `measure/m02_halflife.rs`

Flag: `--measure-halflife`

Measures the topological stability of prisms by recording edge-snap events — the random removal of one intermediary edge that splits a $K_{2,N}$ into $K_{2,N-1} + K_{2,1}$. The half-life is the Markov hitting time for the first snap event at a given thermal excitation.

Key Result

The electron ($K_{2,3}$) is topologically stable: splitting to $K_{2,2} + K_{2,1}$ requires overcoming the K_5 planarity barrier (energy $\sim M_{Pl}$). The muon and tau have finite half-lives proportional to their genus gap.

Output: `half_life.csv` with per-generation half-lives and snap rates.

9.3 M3: Modulo Path Integral (NTT)

Source: `measure/m03_modulo.rs`

Flag: `--measure-modulo`

Implements the **Number-Theoretic Transform** (NTT) path integral: walkers accumulate action S as $g^S \bmod p$ where g is a primitive root and p is a prime modulus. This replaces the continuum $e^{iS/\hbar}$ with a discrete $\mathbb{F}_p$ phase.

Configuration:

- prime: default $65537 = 2^{16} + 1$ (Fermat prime)
- root: default 3 (primitive root of 65537)
- steps: default 500 NTT walk steps

The interference pattern emerges from modular cancellation: when $\sum g^{S_j} \equiv 0 \pmod{p}$, the path is destructively forbidden.

Output: *modulo_interference.csv* with phase histograms and interference fringe visibility.

9.4 M4: Vacuum Polarisation

Source: *measure/m04_vacuum_pol.rs*

Flag: `--measure-vacuum`

Measures the running of the topological charge Q_{topo} with scale. At short distances (small walker steps), the effective coupling increases due to virtual prism–anti-prism screening:

$$\alpha_{\text{EM}}^{-1}(0) \approx 132 \quad \rightarrow \quad \alpha_{\text{EM}}^{-1}(m_Z) \approx 128.$$

The mechanism is planarity screening: at short distances, prism–anti-prism pairs are resolved and screen the effective charge.

Output: *vacuum_polarization.csv* with $Q_{\text{topo}}(t)$ and $\alpha^{-1}(t)$ at each scale t .

9.5 M5: Electroweak Sector

Source: *measure/m05_electroweak.rs*

Flag: `--measure-electroweak`

Probes the separation of $SU(2)_L \times U(1)_Y$ from the prism topology. This measurement uses the asymmetry between left-handed and right-handed walker trajectories to extract the Weinberg angle θ_W :

$$\sin^2 \theta_W = \frac{\text{left-blocked}}{\text{left-blocked} + \text{right-blocked}}.$$

Output: *electroweak.csv* with mixing angle and left/right asymmetry.

Measurements M6–M10

Chapter Overview

- *M6: Quantum decoherence and Born rule*
- *M7: Neutrino census*
- *M8: PMNS mixing matrix*
- *M9: Higgs mechanism (topological drag)*
- *M10: Full Standard Model Lagrangian card*

10.1 M6: Quantum Decoherence

Source: `measure/m06_decoherence.rs`

Flag: `--measure-decoherence`

Tests the emergence of the Born rule from modular path-integral interference. Walkers perform NTT paths and the probability distribution is compared to the quadratic law:

$$P(d) = |K(s, d)|^2,$$

where $K(s, d)$ is the modular propagator from source s to detector d . The double consistency (retarded $\times$ advanced) of the discrete path integral naturally produces the Born rule without postulation.

Output: `decoherence.csv` and `born_rule.csv` with measured vs. predicted detection probabilities.

10.2 M7: Neutrino Census

Source: `measure/m07_neutrino.rs`

Flag: `--measure-neutrino`

Counts “open prisms” — $K_{2,1}$ structures (two poles, one intermediary) that are topologically sterile ($\Phi = 0$) and have near-zero topological mass. These are identified as neutrinos: gravitationally active but electromagnetically invisible.

The census provides per-generation neutrino counts and the neutrino/charged-lepton ratio.

Output: `neutrino.csv` with generation-resolved neutrino counts.

10.3 M8: PMNS Mixing Matrix

Source: `measure/m08_pmns.rs`

Flag: `--measure-pmns`

Requires M7 (automatically enabled). Extracts the PMNS mixing matrix from neutrino oscillation probabilities measured via walker transmission between generation-tagged open prisms. The mixing angles ($\theta_{12}, \theta_{13}, \theta_{23}$) are extracted from the matrix elements.

Key Result

FEG predicts “anarchy”-style PMNS mixing: large mixing angles emerging from the random topology of the Hasse diagram, in agreement with the observed near-maximal θ_{23} and large θ_{12} .

Output: `pmns.csv` with the 3×3 mixing matrix and extracted angles.

10.4 M9: Higgs Mechanism

Source: *measure/m09_higgs.rs*

Flag: `--measure-higgs`

Models the Higgs boson as a “breathing mode” of the Causal Prism ensemble: a coherent oscillation of the average belly size $\langle N \rangle$ about its equilibrium value. The entropy cost $\Delta S \sim 1/\alpha$ of a uniform shift in N sets the Higgs mass scale:

$$m_H \sim v\sqrt{2\lambda}, \quad \lambda \sim \alpha^2.$$

The measurement applies topological drag — walkers on prism components experience an effective potential proportional to their belly size, and the oscillation frequency is extracted from the autocorrelation.

Output: *higgs.csv* with breathing-mode frequency, effective λ , and mass estimate.

10.5 M10: Standard Model Lagrangian Card

Source: *measure/m10_lagrangian.rs*

Flag: `--measure-lagrangian`

Assembles a complete Standard Model parameter card from M1–M9 outputs:

Parameter	Source	FEG Derivation
α_{EM}	M4	$Q_{\text{topo}}/(8\pi)$
$\sin^2 \theta_W$	M5	Left/right walker asymmetry
$m_e : m_\mu : m_\tau$	M1	Cover-time ratios
PMNS matrix	M8	Neutrino walker transmission
m_H	M9	Breathing-mode frequency
τ_μ, τ_τ	M2	Edge-snap hitting times
α_s	M4	Short-distance Q_{topo} running
$P = K ^2$	M6	Modular path integral
Neutrino counts	M7	Open-prism census

This is a post-hoc assembly: M10 does not run its own simulation, but collects and cross-validates the outputs of M1–M9.

Output: *lagrangian.csv* with the full parameter card.

Output & CSV

Chapter Overview

- *CSV serialisation with provenance headers*
- *36-column spectral CSV format*
- *Topology summary and mass spectrum CSVs*
- *Human-readable terminal summary*

11.1 CSV Writer

Source: `output/csv.rs`

The `CsvWriter` struct wraps a `BufWriter<File>` and prepends the standard provenance header block (Layer 3):

```
# FEG Kuratowski Calculus Engine (FEG_prism)
# Author: Juan Pablo Silva Alvarado
# DOI: 10.5281/zenodo.18746995
# Framework: Fractal Entropic Geometrodynamics
# Provenance SHA-256: a4fee337...
# Commit: 5b07174 (dirty)
# Timestamp: 2026-02-24T12:00:00Z
```

11.2 Spectral CSV

Source: `output/mod.rs`

The function `write_spectral_csv()` produces `results.csv` with 36 columns per diffusion step:

Column(s)	Description
step	Diffusion time t (integer steps)
P_vac, dS_vac	Vacuum return probability and spectral dimension
P_def, dS_def	Defect (post-contraction) return prob. and d_S
P_Gen1..3, dS_Gen1..3	Per-generation walks
P_Anti1, dS_Anti1	Antimatter generation walk
Flux_Attr, Flux_Repu	Causal flux (raw)
Flux_Attr_Norm, Flux_Repu_Norm	Normalised flux per target
P_Sterile, dS_Sterile	Sterile prism walk
Mass_Gen1..3, Mass_Anti1	Average topological mass
dS*_std	Ensemble standard deviations
Flux*_std	Flux ensemble standard deviations

11.3 Topology CSV

The function `write_topology_csv()` writes `topology_summary.csv` with all 27 `TopologySummary` fields. The function `write_mass_spectrum_csv()` writes `mass_spectrum.csv` with the prism histogram (belly size $\rightarrow$ count).

11.4 Measurement CSVs

The function `measure::write_all_csv()` dispatches to per-measurement `write_csv()` functions. Output files:

- *traversal_mass.csv* (M1)
- *half_life.csv* (M2)
- *modulo_interference.csv* (M3)
- *vacuum_polarization.csv* (M4)
- *electroweak.csv* (M5)
- *decoherence.csv*, *born_rule.csv* (M6)
- *neutrino.csv* (M7)
- *pmns.csv* (M8)
- *higgs.csv* (M9)
- *lagrangian.csv* (M10)

11.5 Terminal Summary

Source: *output/summary.rs*

The `print_summary()` function writes a human-readable report to stdout including: spectral dimension at key scales, mass hierarchy ratios, prism counts, α_{EM} , dark matter fraction, and measurement highlights.

11.6 Metadata

The Metadata struct accompanies every CSV file:

- `n_points`, `actual_m`, `converged`
- `min_ensemble`, `max_ensemble`
- `mode` (“in-memory” or “streaming”)
- `epsilon`, `tmax`, `walkers`
- `seed`, `timestamp`, `commit`

Animation Export

Chapter Overview

- *Binary topology slice for Manim scenes*
- *Compact bincode serialisation*
- *Integration with the animation layer*

12.1 Topology Slice

Source: `anim_export.rs`

The animation export produces a single-realisation snapshot containing:

Definition 1: TopologySlice

- `n_total`: number of spacetime events
- `coordinates`: $[t, x, y, z]$ for each event
- `hasse_edges`: directed (u, v) Hasse links
- `prisms`: vector of `PrismDef` (origin, destination, intermediates)

The slice is serialised via bincode to a compact binary file. Typical size: ~ 1 MB for $N = 5000$.

12.2 Usage

Invoked by the `--export-slice` flag:

```
feg_prism 5000 1 ./output --inmemory \
  --export-slice ./output/slice.bin --seed 42
```

This runs a single realisation (Phases 1–2 only, no spectral walkers) and writes the topology slice. The Python animation layer (`animations/causal_anim/scene.py`) loads the slice and renders:

- 3D Hasse diagram with prism highlighting
- Time-evolution of causal structure
- Generation-coloured prism components

Configuration

Chapter Overview

- TOML configuration file `causal_set.toml`
- RAM-aware mode selection
- Memory estimation formulas

13.1 Configuration File

Source: `config.rs`

The engine reads `causal_set.toml` from the output directory or current directory. If absent, a default is created:

```
# FEG Prism Simulation - configuration
output_dir = "."
max_ram_gb = 0          # 0 = auto-detect
safety_fraction = 0.70
```

13.2 Config Struct

Definition 1: Config Fields

- `max_ram_bytes`: hard RAM limit (0 = use system detection)
- `safety_fraction`: fraction of available RAM considered safe (default 0.70, clamped to [0.05, 1.0])
- `output_dir`: default output directory (CLI overrides)

13.3 Memory Estimation

Definition 2: Memory Formulas

In-memory mode:

$$\text{est}(N) = \begin{cases} N^2 \times 24 & N \leq 3000 \text{ (eigendecomp: dense matrix)} \\ N \times 2000 & 3000 < N \leq 50,000 \\ N \times 500 & N > 50,000 \text{ (Hasse degree } \leq 15) \end{cases}$$

Streaming mode:

$$\text{est}(N) = N \times 13 + 100 \text{ MB}$$

The recommendation engine (`recommend_mode()`) computes $\text{est}(N) \times \text{concurrent}$ and compares to $\text{available RAM} \times \text{safety_fraction}$.

13.4 Mode Selection Flow

1. If `--inmemory`: force in-memory.
2. If `--stream`: force streaming.
3. Otherwise: auto-detect via `recommend_mode()`. Display memory summary and prompt user to confirm or override.

Provenance

Chapter Overview

- *Five-layer SHA-256 provenance system*
- *No external crate for SHA-256 (custom implementation)*
- *Binding the binary to its author, DOI, and git commit*

14.1 Architecture

Source: `provenance.rs`

The provenance system provides a chain of trust from source code to output files:

Layer	When	Purpose
1	Compile	<code>#[test]</code> canary: <code>sha256(PREIMAGE) == HASH</code>
2	Startup	Runtime verification: <code>verify_provenance()</code>
3	Output	CSV header stamps with author, DOI, hash, commit
4	Resume	Checkpoint provenance: reject cross-fork resume
5	Build	<code>env!()</code> embedding: git hash + dirty flag

14.2 Preimage and Hash

The provenance preimage is the ASCII string:

```
Juan Pablo Silva Alvarado:10.5281/zenodo.18733424:
FEG-Kuratowski-2026
```

Its SHA-256 hash is:

```
a4fee33737305b5eb78a16be2ff233bd
ce125bffa352127fcbcd997d018c05aa
```

Both the preimage and hash are compiled into the binary as constants.

14.3 Custom SHA-256

The engine implements SHA-256 from scratch without external crates (204 lines). The implementation:

- Supports messages up to 119 bytes (sufficient for the preimage)
- Follows FIPS 180-4 exactly: padding, schedule expansion, 64-round compression with the standard K constants
- Verified by the compile-time canary and two additional unit tests (`hex_matches_bytes`, `runtime_verify`)

14.4 Build-Time Embedding

The `build.rs` script (not shown) sets two environment variables:

- `GIT_HASH`: short git commit hash
- `GIT_DIRTY`: “dirty” or “clean”

These are embedded via `env!()` macros and surfaced by `commit_string()`: e.g., "5b07174 (dirty)".

14.5 UTC Timestamp

The function `utc_timestamp()` computes the current time without external crates, using `SystemTime::now()` and manual calendar arithmetic (leap-year-aware). Format: ISO 8601 (2026-02-24T12:00:00Z).

Usage & CLI Reference

Chapter Overview

- Complete CLI reference with all flags
- Quick-start examples
- Common workflows

15.1 Synopsis

```
feg_prism [N] [M] [output_dir] [FLAGS...]
```

15.1.1 Positional Arguments

Argument	Default	Description
N	5000	Number of spacetime events
M	50	Maximum ensemble realisations (or use <code>--max-ensemble</code>)
output_dir	.	Directory for all output files

15.2 Flags

15.2.1 Execution Mode

Flag	Description
<code>--inmemory</code>	Force in-memory mode (fast)
<code>--stream</code>	Force streaming mode (low RAM)
<code>--resume</code>	Resume from last checkpoint

15.2.2 Physics Parameters

Flag	Default	Description
<code>--epsilon <f64></code>	0.01	Topological error tolerance ε
<code>--tmax <usize></code>	15	Maximum diffusion time
<code>--seed <u64></code>	clock	Base seed (deterministic mode)
<code>--eigen-cutoff <N></code>	3000	Eigendecomp threshold

15.2.3 Ensemble Control

Flag	Default	Description
<code>--max-ensemble <N></code>	50	Safety cap on realisations
<code>--min-ensemble <N></code>	8	Minimum before convergence check
<code>--target-error <F></code>	0.01	Relative SE target
<code>--batch-size <N></code>	4	Max concurrent per batch

```
--threads <N>          auto          Max parallel realisations
```

15.2.4 Measurements

Flag	Measurement
--measure-mass	M1: Traversal mass ratios
--measure-halflife	M2: Half-life census
--measure-modulo	M3: Modulo path integral
--measure-vacuum	M4: Vacuum polarisation
--measure-electroweak	M5: Electroweak sector
--measure-decoherence	M6: Quantum decoherence
--measure-neutrino	M7: Neutrino census
--measure-pmns	M8: PMNS mixing matrix (enables M7)
--measure-higgs	M9: Higgs mechanism
--measure-lagrangian	M10: Full SM Lagrangian card (enables M1–M9)
--measure-all	Enable all M1–M9

15.2.5 Modulo Configuration

Flag	Default	Description
--modulo-prime <u64>	65537	Prime modulus p for M3/M6
--modulo-root <u64>	3	Primitive root g of p
--modulo-steps <N>	500	NTT walk steps

15.2.6 Export

Flag	Description
--export-slice <PATH>	Export topology slice (single realisation, early exit)
--help	Show usage help

15.3 Quick-Start Examples

15.3.1 Minimal Run

```
cargo run --release --bin feg_prism -- \
  5000 8 ./output --inmemory --seed 42
```

Runs $N = 5000$, $M \leq 8$ realisations, deterministic seed, in-memory mode.

15.3.2 Full Standard Model Card

```
cargo run --release --bin feg_prism -- \
  50000 25 ./sm_run --inmemory \
  --measure-lagrangian --seed 42
```

Enables all M1–M10 measurements and produces the complete SM parameter card.

15.3.3 Production Run

```
cargo run --release --bin feg_prism -- \
  1000000 50 ./production --inmemory \
  --measure-all --target-error 0.005 --seed 42
```

One million events, up to 50 realisations, tighter convergence target.

15.3.4 Born Rule Verification

```
cargo run --release --bin feg_prism -- \  
5000 10 ./born --inmemory \  
--measure-decoherence --seed 42
```

15.3.5 PMNS Mixing Matrix

```
cargo run --release --bin feg_prism -- \  
50000 25 ./pmns --inmemory \  
--measure-pmns --seed 42
```

Automatically enables M7 (neutrino census) as a prerequisite.

15.3.6 Higgs Mass Estimate

```
cargo run --release --bin feg_prism -- \  
50000 25 ./higgs --inmemory \  
--measure-higgs --seed 42
```

15.3.7 Animation Export

```
cargo run --release --bin feg_prism -- \  
5000 1 ./anim --inmemory \  
--export-slice ./anim/slice.bin --seed 42
```

Early exit after Phase 2; no spectral walkers run.

15.3.8 Resume from Checkpoint

```
cargo run --release --bin feg_prism -- \  
50000 50 ./production --inmemory \  
--measure-all --resume --seed 42
```

Continues from the last checkpoint in *./production/*.

15.3.9 Custom Modulo Parameters

```
cargo run --release --bin feg_prism -- \  
5000 10 ./modulo --inmemory \  
--measure-modulo --modulo-prime 257 \  
--modulo-root 3 --modulo-steps 1000 --seed 42
```

Expected Results & Verification

Chapter Overview

- Expected output values at standard scales
- Verification checklist
- Known finite-size effects

16.1 Spectral Dimension

Key Result

At $N = 50,000$, $M = 25$:

- $d_S(t \rightarrow 0) \rightarrow 2$ (short-distance fractal)
- $d_S(t \rightarrow \infty) \rightarrow 4$ (long-distance manifold recovery)
- The crossover occurs at $t \sim 5$ – 10 diffusion steps
- Defect d_S is slightly reduced compared to vacuum (matter introduces topological shortcuts)

16.2 Mass Hierarchy

Key Result

Cover-time mass ratios ($M1$, $N = 50,000$, $M = 25$):

- $m_\mu/m_e \approx 200 \pm 30$ (target: 206.77, PDG)
- $m_\tau/m_e \approx 3500 \pm 500$ (target: 3477.4, PDG)

Exact agreement improves with N ; finite-size corrections scale as $N^{-1/4}$.

16.3 Fine Structure Constant

Key Result

Phase-coherence coupling ($N = 50,000$, $M = 25$):

- $Q_{\text{topo}} = \sum |\Phi|^2 / \sum N^2 \approx 0.06$ – 0.08
- $\alpha_{EM} = Q_{\text{topo}} / (8\pi) \approx 1/132$ at bare scale
- Running via planarity screening: $\alpha^{-1} \rightarrow 137$ at long distances

16.4 Dark Matter Fraction

Key Result

Phase-coherence decomposition ($N = 50,000$, $M = 25$):

- $\Omega_{\text{dark}}/\Omega_{\text{vis}} \approx 4$ – 6 (target: 5.3, Planck)
- Sterile prisms ($\Phi = 0$) constitute ~ 10 – 20% of all prisms

16.5 Born Rule

Key Result

Decoherence measurement ($M6$, $N = 5,000$, $M = 10$):

- $P(d) \propto |K(s, d)|^2$ verified to within 5%
- Fringe visibility > 0.8 at $t < 10$; decoheres for $t > 50$

16.6 Verification Checklist

1. Compile: `cargo build --release --bin feg_prism`
2. Provenance: startup message shows “SHA-256 verified ✓”
3. Triangle-freeness: `topology_summary.csv` shows zero K_4 count
4. Prism counts: $\sim 5\text{--}15\%$ of core nodes form prisms
5. Generation hierarchy: $\text{Gen1} > \text{Gen2} > \text{Gen3}$ in abundance
6. Spectral dimension: $d_S(1) \approx 2$, $d_S(30) \approx 4$
7. CSV files: all provenance headers present and hash matches
8. Determinism: same seed produces identical output

16.7 Known Finite-Size Effects

- **Boundary effects:** The causal diamond has sharp boundaries. Nodes near the tips have anomalously low degree. The core selection (top 10% by degree) mitigates this by preferring interior nodes.
- **Mass ratio convergence:** The cover-time ratios m_μ/m_e and m_τ/m_e converge slowly (as $N^{-1/4}$). For precision to within 10%, use $N \geq 50,000$.
- **Spectral dimension plateau:** The $d_S \rightarrow 4$ plateau may not be reached at small $t_{\max}$. Use $t_{\max} \geq 15$ for convergence.
- **Ensemble variance:** Some observables (e.g., α_{EM}) have high variance across realisations. Use $M \geq 25$ for reliable estimates.

Dependencies

Chapter Overview

- *Nine Rust crates*
- *No external SHA-256 dependency*
- *Build profile for performance*

17.1 Crate List

Crate	Version	Purpose
<code>nalgebra</code>	0.33	Dense linear algebra (eigendecomposition)
<code>sprs</code>	0.11	Sparse matrix operations (A^2 for Tier 1 Hasse)
<code>rand</code>	0.8	Pseudorandom number generation (<code>StdRng</code>)
<code>rayon</code>	1.10	Data parallelism (parallel iterators)
<code>sysinfo</code>	0.33	System memory detection
<code>smallvec</code>	1.15.1	Stack-allocated small vectors
<code>rustc-hash</code>	2	Fast non-cryptographic hashing
<code>serde</code>	1	Serialisation framework (with <code>derive</code>)
<code>bincode</code>	1.3	Binary serialisation for animation export

17.2 Build Profile

```
[profile.release]
opt-level = 3
lto = "thin"
codegen-units = 1
```

This maximises performance at the cost of longer compile times. The `codegen-units = 1` setting enables whole-program optimisation.

17.3 Notable Absences

- **No SHA-256 crate:** the 204-line custom implementation in *provenance.rs* avoids any external cryptographic dependency.
- **No TOML parser:** the configuration reader in *config.rs* uses simple line-by-line parsing (12 lines).
- **No logging framework:** diagnostics use `println!` with optional file-append to *simulation.log*.

Colophon

This document was typeset with XeLaTeX using the Modulo Synthesis palette and tcolor-box theorem environments, matching the styling of the pedagogic booklet series *Modulo Synthesis, Volumes I & II*.

Source code: https://github.com/ertwro/fractal_entropic_geometrodynamics

DOI: <https://doi.org/10.5281/zenodo.18746995>

License: MIT

© 2026 Juan Pablo Silva Alvarado. All rights reserved.